```
      classifier = "test-binder"
    }
  }
}
```

Next we need to model the event we'd like Order Service to listen to. Create a new `com.polarbookshop.orderservice.order.event` package, and add an `OrderDispatchedMessage` class to hold the identifier for a dispatched order.

Listing 10.18 A DTO representing the event about orders being dispatched

```
package com.polarbookshop.orderservice.order.event;

public record OrderDispatchedMessage (
  Long orderId
){}
```

Now we'll implement the business logic using a functional approach. Create an `OrderFunctions` class (`com.polarbookshop.orderservice.order.event` package), and implement a function to consume the messages produced by the Dispatcher Service application when an order is dispatched. The function will be a `Consumer` responsible for listening to the incoming messages and updating the database entities accordingly. `Consumer` objects are functions with input but no output. To keep the function clean and readable, we'll move the processing of `OrderDispatchedMessage` objects to the `OrderService` class (which we'll implement in a minute).

Listing 10.19 Consuming messages from RabbitMQ

```
package com.polarbookshop.orderservice.order.event;

import java.util.function.Consumer;
```